

Ever wondered how hackers break into 'unbreakable' software without ever seeing the source code?

Today, we stop being users and start becoming architects of the digital underworld.

What is Reverse Engineering?

- Process of analyzing software to understand how it works
- Starts from compiled program, not source code
- Breaks software into smaller parts for study
- Reveals logic, behavior, and hidden functionality
- Simply: working backward from the final product



Goals of RE & Use Cases

- Analyze malware behavior and capabilities
- Discover security bugs and vulnerabilities
- Understand undocumented software features
- Improve or patch existing software
- Ensure compatibility between systems
- Support digital forensics and incident response



The Compilation Process

- Developers write Source Code (C, C++, etc.)
- Compiler converts it to Assembly Language
- Assembler converts it to Machine Code (binary)
- CPU executes only machine code
- Reverse Engineering works backward in this chain

Forward vs Reverse Engineering

- **Forward Flow:**
- Source Code → Assembly → Machine Code
- **Reverse Engineering Flow:**
- Machine Code → Assembly → (Approx) Source Code
- **Disassembler:** Machine Code → Assembly
- **Decompiler:** Assembly → High-level Source (approximate)

Introduction to Assembly Language

- Low-level, human-readable form of machine instructions
- Directly controls CPU operations
- Harder than high-level languages
- Main language studied in reverse engineering
- Shows what program actually does internally



CPU & Memory Basics

Registers:

- Very small and extremely fast storage inside CPU
- Hold temporary values during execution
- Frequently used in assembly analysis

Important Registers:

- **EAX:** Often used for calculation results
- **EIP:** Most critical — points to the next instruction to execute

The Stack

- Memory structure used during function calls
- Stores return addresses and local variables
- Works on LIFO (Last In, First Out)

Static vs Dynamic Analysis

Legal & Ethical Considerations



Golden Rules for Safe Practice